Rich Fox

ITAI 2373

August 29, 2026

Reflection Journal – Lesson 02

This lesson gave me a better understanding of how important text preprocessing is to Natural Language Processing (NLP). Before working through these concepts, I thought preprocessing was mostly about cleaning up text so that a computer could read it more easily. I now understand that the choices made during preprocessing can actually affect what information an NLP system is able to understand. Removing words, punctuation, emojis, or changing words through stemming can make the data easier to process, but it can also remove information that may be important to the final results.

One of the biggest insights I gained was that there is not one preprocessing approach that is best for every NLP application. The comparison between NLTK and spaCy helped demonstrate this. Both libraries were able to tokenize the sample text, but spaCy provided additional linguistic information such as parts of speech, lemmas, and stop-word identification. I can see why this would make spaCy useful for applications that need more understanding of the structure and meaning of text. At the same time, NLTK's simpler approach could be useful when I need more control over the individual preprocessing steps.

I also found the difference between stemming and lemmatization interesting. Stemming can be fast and useful for reducing words to a common form, but some of the results were not actual English words. For example, words such as "studies" becoming "studi" and "was" becoming "wa" show that stemming can be fairly aggressive. Lemmatization produced more meaningful results because it considers the actual word and its linguistic form. This made me realize that faster processing is not always better if it causes important meaning to be lost.

The main technical challenge I encountered was an error when using NLTK tokenization because the punkt_tab tokenizer data was missing. I initially had a LookupError when calling the tokenizer. I had to download the missing resource using nltk.download('punkt_tab') before the tokenizer would work correctly. Although this was a relatively small problem, it was a useful reminder that NLP libraries often depend on additional resources that need to be installed or downloaded. Troubleshooting this also gave me more confidence in understanding that an error does not necessarily mean the code itself is wrong.

I made several connections between these techniques and real-world applications. Customer reviews are a good example because sentiment can depend on information that might normally be removed during preprocessing. A review such as "This laptop is fantastic!!! 😍" contains more emotional information than simply the words "laptop" and "fantastic." Similarly, hashtags and emojis can provide useful information in social media analysis. This makes me think that an NLP system analyzing customer reviews should probably use moderate preprocessing instead of aggressively removing everything that appears unnecessary.

One question I still have is how professional NLP systems decide exactly which preprocessing techniques to use. For example, how does a developer determine whether an emoji, punctuation mark, or stop word should be removed? I also wonder how much these decisions depend on the specific dataset being used. It seems likely that preprocessing that works well for news articles may not work as well for social media posts or customer reviews.

In future projects, I could see myself using these techniques for analyzing customer feedback, classifying documents, or building a system that identifies sentiment from text. I would probably start with a relatively conservative preprocessing approach and then test whether removing additional information actually improves the results. Overall, this lesson showed me that preprocessing is not just a cleanup step. It is an important part of designing an NLP system because the decisions made before the model is trained can have a major impact on what the model ultimately learns.